

3D DATA PROCESSING - LAB 2

Structure from Motion - Report

Task 1: Geometric Verification

In `exhaustiveMatching()`, we implemented geometric verification to filter raw feature matches. After performing an initial Brute-Force match to identify candidates based on descriptor similarity, we used OpenCV's `findEssentialMat()` and `findHomography()` with RANSAC to estimate the epipolar and planar geometry between image pairs. Matches are only promoted to "inliers" if they satisfy the Essential matrix model, and we require at least 8 matches to consider a pair valid for the scene graph.

Task 2: Modern Feature Extraction

We integrated the requested modern feature detection scheme using **SuperPoint**. We initially **attempted to implement Option A** by loading a pre-trained SuperPoint model in `.onnx` format using `cv::dnn::readNet()`. However, we encountered compatibility issues between the ONNX version of the model and the OpenCV build available in our environment. To overcome this problem, we **switched to Option B**. We wrote a dedicated Python script to handle the part of deep learning inference.

- **External Extraction Script:** We utilized a dedicated script, `extract_superpoint.py`, to handle the deep learning inference. The script loads the **pre-trained SuperPoint weights** from `superpoint_v1.pth` (downloaded from the provided GitHub repository) using the PyTorch framework. These two files can be found in the `modern_features` folder.
- **Preprocessing and Undistortion:** Before feature extraction, the script reads the `.yaml` calibration file to undistort the images, ensuring the keypoints are extracted from the canonical perspective required by our SfM pipeline.
- **Feature Export:** The script runs the `SuperPointNet` model to detect salient points and compute 256-dimensional descriptors, saving them as `.txt` files in a `features/` directory.

To run the script directly from the build directory (example for `datasets/images_1`):

```
python ../modern_features/extract_superpoint.py
../modern_features/superpoint_v1.pth ../datasets/images_1
../datasets/3dp_cam.yaml 1.1
```

- **C++ Data Loading:** We implemented the `loadExternalFeatures()` function in `features_matcher.cpp` to parse these text files, loading the keypoints into the `features_vector` and the descriptors into a `cv::Mat`.
- **Color Assignment:** Because SuperPoint does not natively capture color, we added logic in `extractFeatures()` to look up the RGB values from the undistorted images at each keypoint location to populate the `feats_colors_vector`.

Task 3: Seed Pair Selection

Inside `incrementalReconstruction()`, we implemented logic to find a suitable starting pair for the 3D model.

- **Model Comparison:** We ensure the number of inliers for the Essential matrix (**E**) is higher than for the Homography matrix (**H**), this ensures that the two images were taken from different positions in space rather than just being a flat 2D rotation or a view of a single flat surface.
- **Cheirality & Motion:** We use `cv::recoverPose()` to extract the initial Rotation (**R**) and Translation (**t**). We specifically check for "sideward motion" by ensuring the forward translation (**tz**) does not dominate the total translation, which leads to more stable triangulation.

Task 4: Point Triangulation

For each newly registered camera, we triangulate new 3D points by finding correspondences in previously registered images.

- **Projection Matrix Construction:** Before triangulating, we construct 3x4 projection matrices ($P = [R|t]$) for both the newly registered camera and the existing reference camera. Since the cameras are in the canonical perspective, we use the identity matrix for **K**, meaning the projection matrices consist purely of the rotation and translation parameters optimized by the solver.
- **Triangulation Method:** We use `cv::triangulatePoints()`, which implements a Linear Least Squares method to find the 3D intersection of the two 2D rays. The resulting homogeneous coordinates are then normalized ($X/W, Y/W, Z/W$) to obtain the final Euclidean 3D coordinates.
- **Cheirality Constraint:** To filter out mathematically valid but physically impossible points, we added a Cheirality Constraint check using the existing helper function `checkCheiralityConstraint()`. This verifies that the triangulated point has a positive Z-depth relative to the coordinate frames of both cameras.
- **Validation Masking:** Only points that pass the cheirality check are marked as active in the `pts_optim_iter_mask`. Any point that fails (meaning it is projected behind a camera) is discarded to prevent "ghost" geometry from ruining the bundle adjustment.

Task 5: Auto-Diff Cost Function

We implemented a `ReprojectionError` struct to define the **optimization objective** for the Ceres Solver.

- **Projection Logic:** The functor uses `AngleAxisRotatePoint()` to rotate a 3D point into the camera's local frame, adds the translation vector, and performs a canonical perspective projection ($x = X/Z, y = Y/Z$).

- **Residuals:** The error is defined as the Euclidean distance between the predicted 2D projection and the actual observed feature point.

Task 6: Bundle Adjustment & Robust Loss Kernel

In `bundleAdjustmentIter()`, we used the Ceres Solver to perform a global optimization of the scene. This step fine-tunes the camera positions and 3D points simultaneously, reducing the "gaps" between where we expect to see a point and where it actually appears in the images. By minimizing this total error, we transform a rough initial guess into a mathematically consistent 3D model.

- **Residual Block Construction:** For every valid 2D-3D correspondence, we added a residual block to the solver using the `ReprojectionError` cost function developed in Task 5.
- **Robust Huber Loss Weighting:** To ensure the model isn't distorted by incorrect matches (outliers), we implemented a Huber loss kernel. By setting the scale to $2.0 * \text{max_reproj_err_}$, the solver treats small errors normally but applies a linear penalty to large errors. This effectively "weights down" suspicious matches, preventing them from pulling the cameras or points out of their correct physical alignment.
- **Solver Configuration:** We encountered an issue during the initial Bundle Adjustment of the seed pair where the Cholesky-based solvers failed. To resolve this, we switched the `linear_solver_type` to `ITERATIVE_SCHUR` with a `JACOBI` preconditioner. Using the iterative solver made the optimization more stable. This was particularly useful for the initial seed pair, as it prevented the system from crashing when the mathematical relationships between points were still uncertain.

Task 7: Divergence Check

To ensure reconstruction stability, we implemented a check at each iteration of the incremental process. We monitor for "exploding" geometry by checking if the camera **translations** or **rotation** magnitudes exceed reasonable thresholds. We also monitor the **Mean Reprojection Error**; if it exceeds 0.5 (normalized), the reconstruction is considered diverged and is reset to start over with a new seed pair.

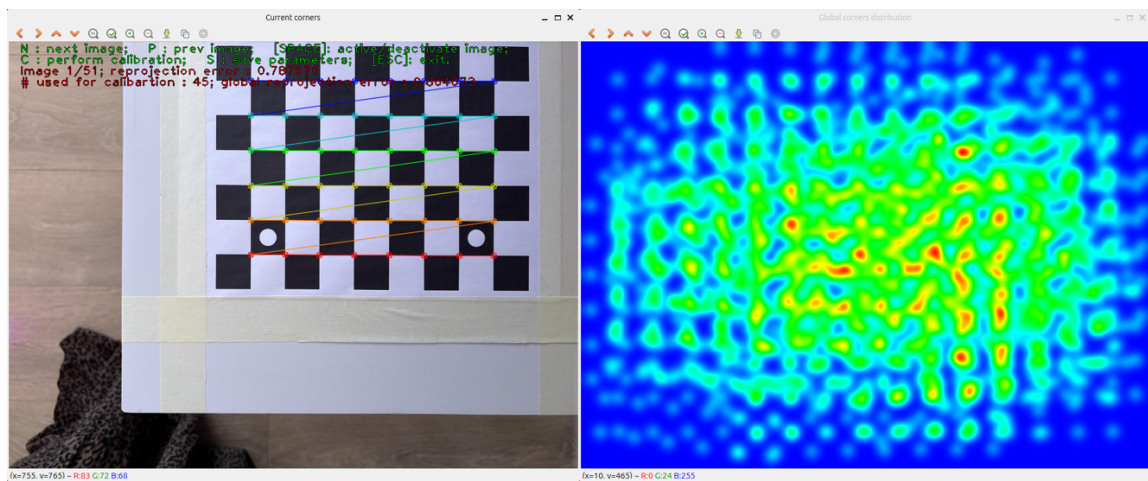
We noticed a couple of things while implementing this task:

- We refactored the pipeline to **prioritize the divergence check over geometry filtering**: by identifying a divergence early, the system can reset and attempt a more viable seed pair rather than attempting to fix a fundamentally broken model by filtering.
- **Reprojection error** was being calculated manually in multiple locations (Divergence Check, Outlier Rejection, and Final Reporting). We refactored this into a single helper function, `computeReprojectionError(int obs_idx)`, to eliminate code redundancy.

Calibration and Datasets

These are the steps we followed to prepare the images and calibration data for the pipeline:

- **Capture Settings:** We captured all photos with an iPhone 16, manually disabling all software corrections and auto-focus to ensure consistent image geometry.
- **Image Processing:** The raw images were 4284x5712. We rescaled these to 1536x2048.
- **Refining the Calibration:** During the chessboard calibration, we monitored the error and the distribution of the corners. We deactivated almost every image with an error greater than 1.0, though we retained a few that provided critical viewpoints as long as they did not exceed the threshold by much. This allowed us to achieve a final global error of 0.86, which was as a reliable **trade-off between the number of views and overall precision**.



- **Focal Length Scaling:** Since the testing datasets were resized to the same pixel dimensions as the calibration chessboard (1536x2048), we used a focal length scale of 1.0 for all subsequent tests.

The following command was used for the calibration process:

```
./bin/cam_calib -f iphone16_1536x2048_chessboard/ -c
iphone16_1536x2048_calib.yml --bw 8 -bh 6 -q 0.027 -k -s 2 -u -opencv
format
```

We selected two subjects with contrasting geometries to test the reconstruction:

- **Tom & Jerry (images_3):** We expected a high density of points because Lego bricks provide sharp corners and distinct edges. However, we also expected potential issues with redundant points due to the repetitive nature of the brick surfaces.
- **Pokémon (images_4):** We expected a less successful reconstruction for this model, specifically for Mew (the pink one, stuffed toy). Its "soft lines" and smooth, curved surfaces offer significantly fewer salient points for the feature extractors to track while Pikachu (3D printed) was expected to perform well.

Workload Division

The project responsibilities were distributed by splitting the work in two parts:

- Data acquisition and modern feature integration.
- Core geometric reconstruction and optimization.

Adelina focused on Data Preparation, Feature Engineering and Testing. Specifically:

- **Task Implementation:** Completed Task 1 (Geometric Verification/RANSAC) and Task 2 (SuperPoint integration).
- **SuperPoint Research and Feature Pipeline:** Developed the Python script for SuperPoint feature extraction and managed the integration of modern descriptors into the C++ environment.
- **Camera and Calibration:** Handled the data collection by managing all photoshoots and performing the camera calibration to generate the required *.yaml* file.
- **Experimental Testing:** Conducted the testing for the pipeline, evaluating performance across the different datasets and comparing the accuracy of ORB versus SuperPoint features, as well as the impact of robust loss kernel against standard least squares.

Antonio focused on SfM Pipeline Development and Optimization. His work involved building the mathematical framework to transform 2D data into a refined 3D model. Specifically:

- **Task Implementation:** Completed Tasks 3, 4, 5, 6, and 7, which covered seed-pair registration, incremental triangulation, the Ceres reprojection cost function, Bundle Adjustment with Huber loss, and the final incremental loop.
- **System Stability:** Identified and resolved critical issues regarding the Cholesky failures by reconfiguring the Ceres solver.

Results

Below are the results of our implementation for each dataset, featuring **performance comparison tables** for each reconstruction test. We have included **screenshots of the generated .ply files** for each test to visualize the final 3D point cloud reconstructions.

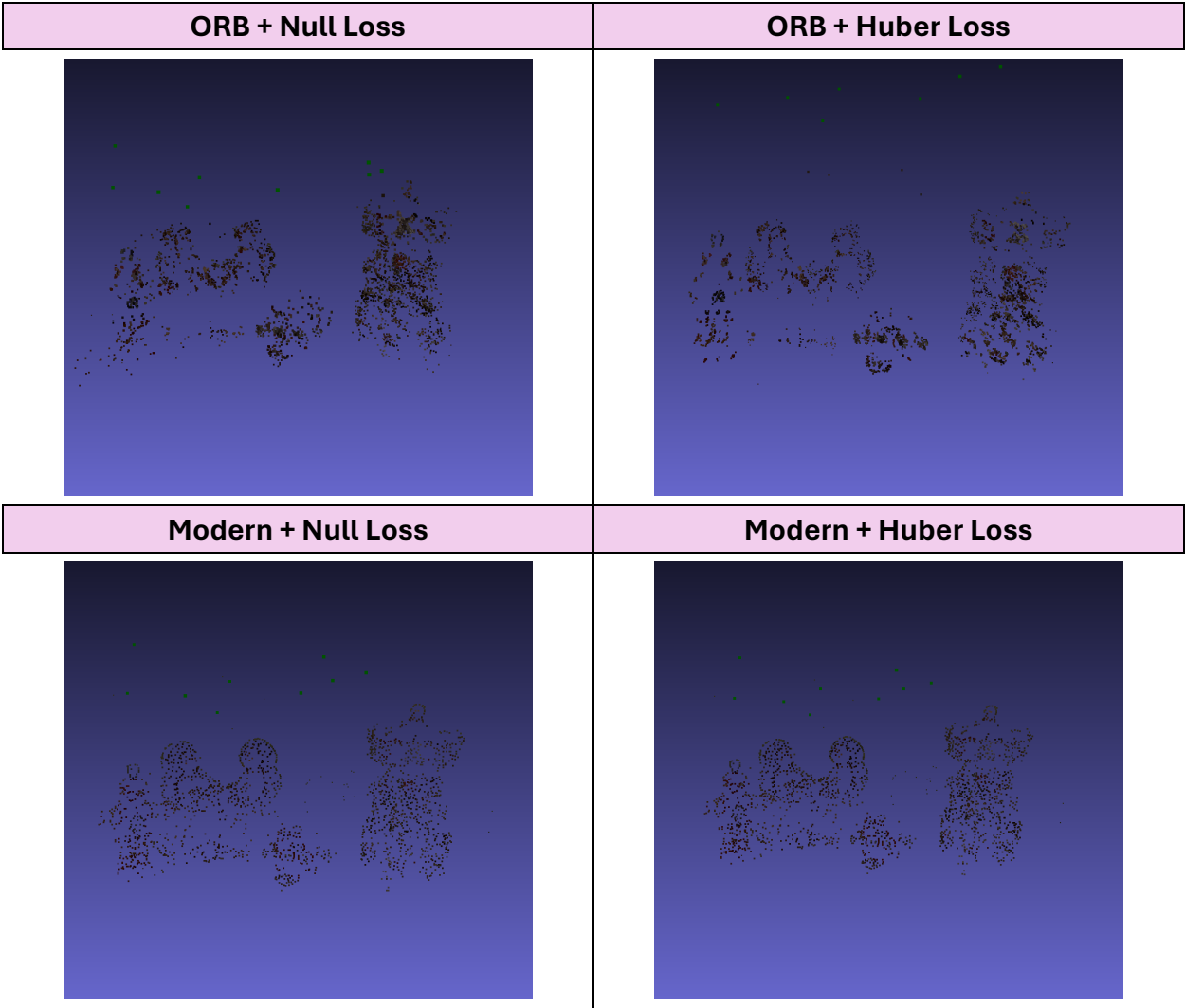
References (original images, in order):



Dataset 1 (Toys)

Configuration	Density [Points]	Success Rate [Cameras]	Average Error [Normalized Pixels]
ORB + Null Loss	3414	9/9	0.00070081
ORB + Huber Loss	4531	9/9	0.00067759
Modern + Null Loss	1852	9/9	0.00096543
Modern + Huber Loss	1861	9/9	0.00095297

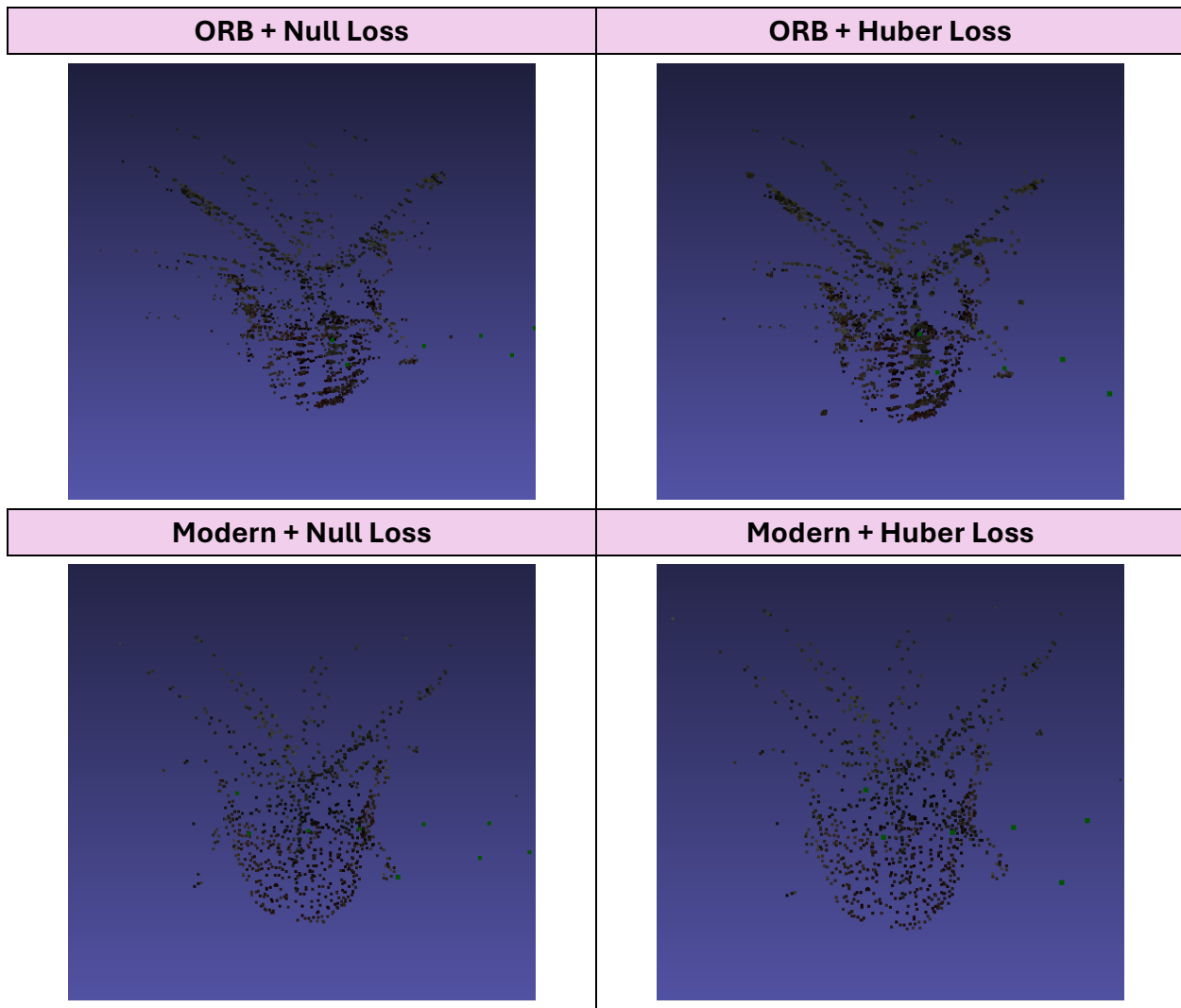
Classical vs Modern Features	ORB	SuperPoint
Total 3D Points initialized	9910	1948
Total Observations recorded	36146	9076



Dataset 2 (Plant)

Configuration	Density [Points]	Success Rate [Cameras]	Average Error [Normalized Pixels]
ORB + Null Loss	2589	15/15	0.00072074
ORB + Huber Loss	3055	15/15	0.00076025
Modern + Null Loss	963	15/15	0.00093825
Modern + Huber Loss	962	15/15	0.00093321

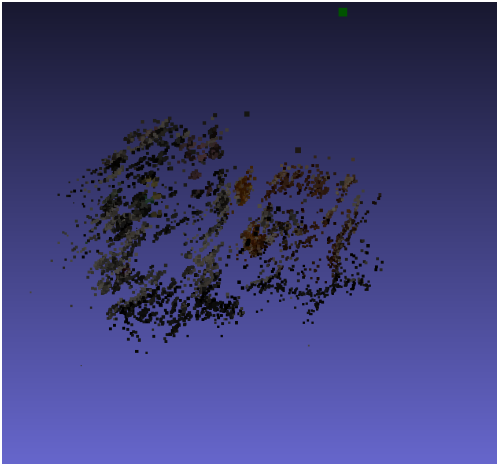
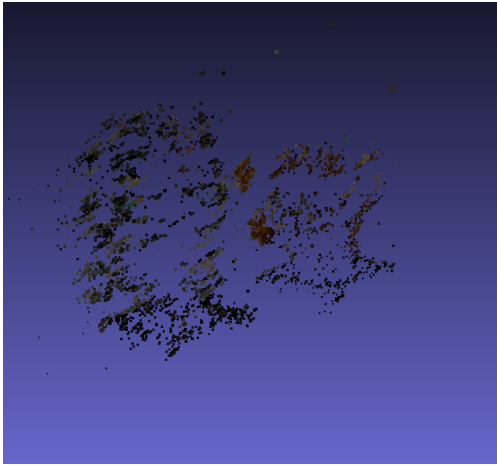
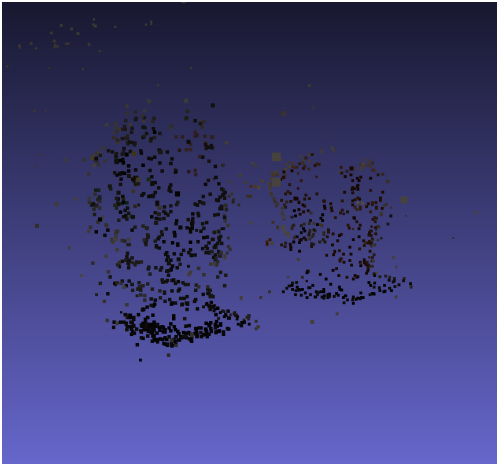
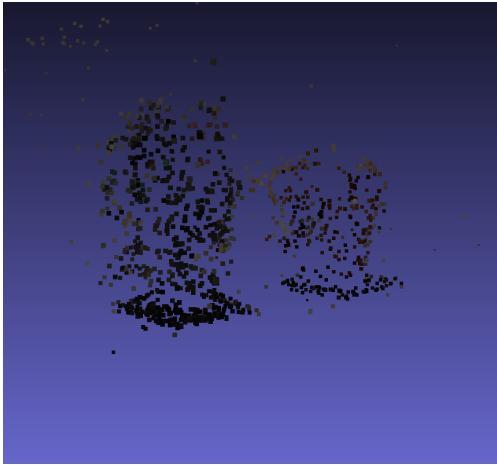
Classical vs Modern Features	ORB	SuperPoint
Total 3D Points initialized	6359	1762
Total Observations recorded	20466	7182



Dataset 3 (Tom & Jerry)

Configuration	Density [Points]	Success Rate [Cameras]	Average Error [Normalized Pixels]
ORB + Null Loss	6587	19/19	0.00139855
ORB + Huber Loss	7475	19/19	0.00129671
Modern + Null Loss	1341	19/19	0.00151052
Modern + Huber Loss	1279	19/19	0.00154317

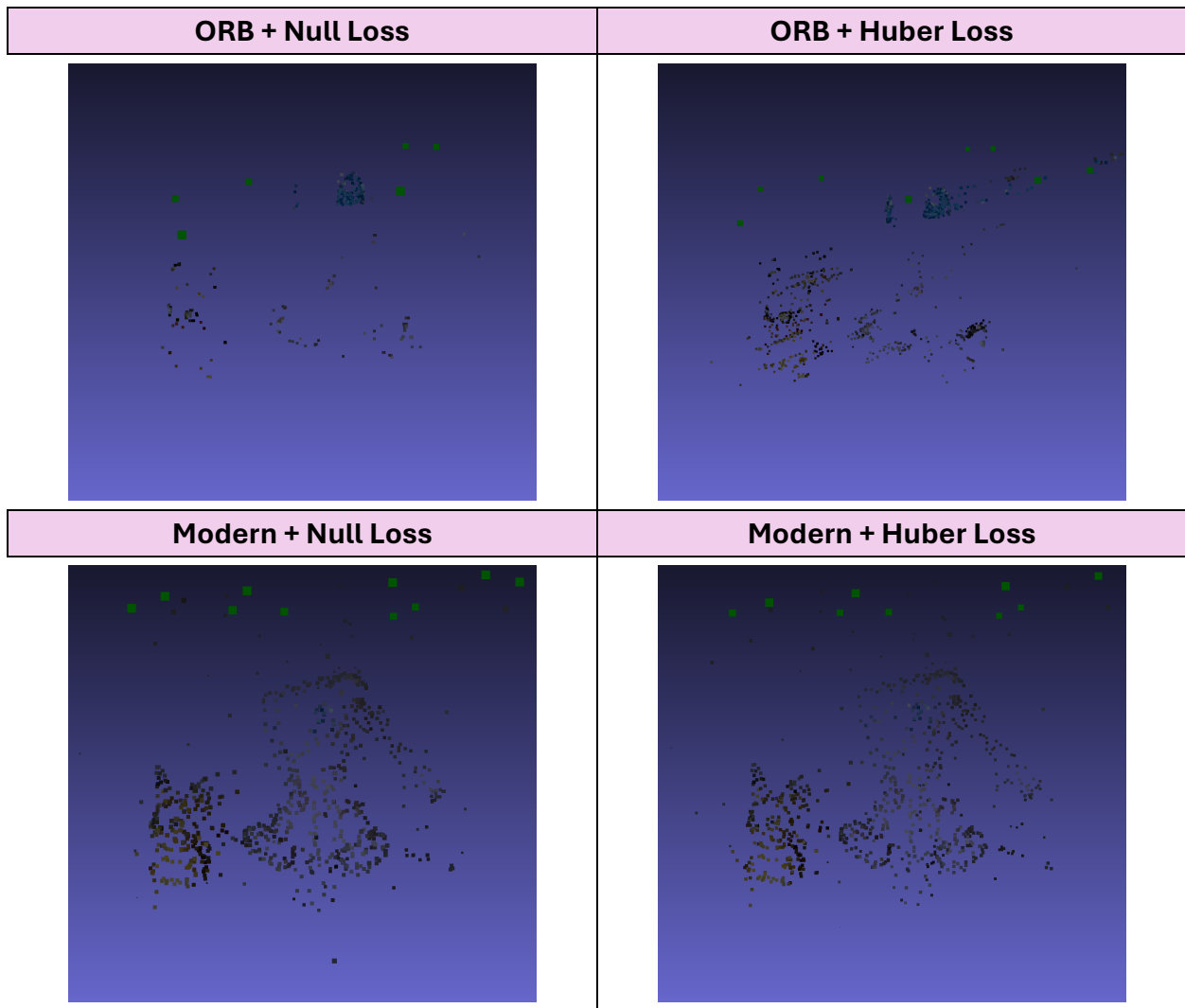
Classical vs Modern Features	ORB	SuperPoint
Total 3D Points initialized	11176	2302
Total Observations recorded	65302	15154

ORB + Null Loss	ORB + Huber Loss
	
Modern + Null Loss	Modern + Huber Loss
	

Dataset 4 (Pokémon)

Configuration	Density [Points]	Success Rate [Cameras]	Average Error [Normalized Pixels]
ORB + Null Loss	376	18/18	0.00093043
ORB + Huber	1269	18/18	0.00068009
Modern + Null Loss	839	18/18	0.00140989
Modern + Huber Loss	868	18/18	0.00138604

Classical vs Modern Features	ORB	SuperPoint
Total 3D Points initialized	4319	1276
Total Observations recorded	17417	5242



Discussion of Results

Density vs. Robustness (Classical vs. Modern)

Across all datasets, ORB (Classical) consistently produced a much higher density of 3D points compared to SuperPoint (Modern).

- **Lego Dataset (Tom & Jerry):** ORB initialized 11,176 points, whereas SuperPoint initialized 2,302. This supports our hypothesis that the sharp corners and edges of Lego bricks are ideal for classical gradient-based detectors.
- **Tracking Consistency:** While ORB creates denser clouds, the Total Observations show that SuperPoint maintains a competitive ratio of observations per point. This suggests that while modern features are sparser, they are highly reliable and successfully tracked across multiple camera poses.

Performance on Challenging Geometry

The Pokémon dataset (specifically the Mew plushie) provided the most critical test for our hypotheses regarding "soft lines" and smooth surfaces.

- **ORB Sensitivity:** In the "Null Loss" configuration, ORB struggled significantly with the plushie's geometry, yielding its lowest point density (376 points) across all test cases.
- **SuperPoint Superiority:** SuperPoint produced 839 points (more than double the count of ORB) and the overall 3D reconstruction was clearly improved. This confirms that deep-learning-based descriptors are superior at identifying salient features on smooth, curved surfaces where traditional corner-detection logic fails.

Accuracy and Reprojection Error

There is a consistent trade-off between feature reliability and sub-pixel precision.

- **Precision:** In every dataset, ORB achieved a lower Average Reprojection Error than SuperPoint. This proves that ORB's mathematical corner localization is more precise in normalized pixel space, whereas SuperPoint's keypoints, while more robust to surface changes, have a slightly higher localization variance.

Impact of Huber Loss

The implementation of the Huber Loss kernel was visibly important in ORB configurations, especially for mitigating outliers.

- **Outlier Mitigation:** The robust loss kernel successfully "weighted down" incorrect matches (outliers), preventing large errors from causing the solver to reject valid 3D points. This allowed the pipeline to recover some geometry on textures that are usually noisy for ORB.

Conclusion

The 100% camera success rate achieved across all tests proves the robustness of the incremental SfM implementation. For high-precision reconstruction of geometric objects with sharp features, ORB with Huber Loss remains the most effective choice due to its high density and low reprojection error. However, for scenes dominated by smooth surfaces or "soft" geometry (like the Mew plushie), SuperPoint is essential for maintaining a viable number of salient points, proving the necessity of modern feature extraction in non-ideal environments.

Personal Reflections

Building this SfM pipeline was a really rewarding experience because of how practical it felt. Moving away from synthetic data and using our own phones to take the photos made a huge difference. It allowed us to see firsthand how real-world factors, like surface textures and calibration accuracy, actually affect the final 3D model. Seeing our own photos turn into solid 3D point clouds made all the math we studied feel much more real and tangible.